Lab 14: Multiple Linear Regression Aim

To implement Multiple Linear Regression on the House Prices dataset and evaluate its performance.

Dataset: House Prices – Advanced Regression Techniques (Kaggle)

Step 1: Import Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Load Dataset

```
# Load dataset
data = pd.read_csv("/content/train (2).csv")

# Select required features
features = ["GrLivArea", "OverallQual", "TotalBsmtSF"]
X = data[features]
y = data["SalePrice"]

# Display first 5 rows
print(X.head())
```

```
   GrLivArea  OverallQual  TotalBsmtSF
0       1710            7          856
1       1262            6         1262
2       1786            7          920
3       1717            7          756
4       2198            8         1145
```

Step 3: Handle Missing Values

```
# Fill missing values with mean
X = X.fillna(X.mean())
```

Step 4: Split Data (Train/Test)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 5: Implement Multiple Linear Regression

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Step 6: Make Predictions

```
y_pred = model.predict(X_test)
```

Step 7: Evaluate Model

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MSE: 1667657527.1633515
RMSE: 40836.96275634797
```

```
R2 Score: 0.7825832328225167
```

Step 8: Regression Coefficients

```
coefficients = pd.DataFrame({
    "Feature": features,
    "Coefficient": model.coef_
})

print(coefficients)
print("Intercept:", model.intercept_)
```

```
       Feature   Coefficient
0    GrLivArea     47.136019
1  OverallQual  28203.610104
2  TotalBsmtSF     33.173543
Intercept: -98427.32987875512
```

Output Interpretation 1️⃣ MSE (Mean Squared Error)

Measures average squared difference between actual and predicted prices.

Lower value ⟶ Better model.

2️⃣ RMSE (Root Mean Squared Error)

Square root of MSE.

Easier to understand because it is in same unit as SalePrice.

3️⃣ R^2 Score

Value between 0 and 1.

Closer to 1 ⟶ Better fit.

Example: $R^2 = 0.75$ means 75% variance explained by model.

Lab 14: Multiple Linear Regression Aim

To implement Multiple Linear Regression on the Insurance dataset and evaluate its performance using MAE, MSE, and R^2 score.

Step-by-Step Implementation (Python)

```
# Step 1: Import Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder

# Step 2: Load Dataset
df = pd.read_csv("/content/insurance.csv")

print(df.head())
```

```
   age     sex     bmi  children smoker     region      charges
0   19  female  27.900         0    yes  southwest  16884.92400
1   18    male  33.770         1     no  southeast   1725.55230
2   28    male  33.000         3     no  southeast   4449.46200
3   33    male  22.705         0     no  northwest  21984.47061
4   32    male  28.880         0     no  northwest   3866.85520
```

Encode Categorical Variables

```
# Label Encoding for categorical columns
le = LabelEncoder()

df['sex'] = le.fit_transform(df['sex'])
df['smoker'] = le.fit_transform(df['smoker'])
df['region'] = le.fit_transform(df['region'])
```

Select Features and Target

```
X = df[['age', 'bmi', 'children', 'smoker']]
y = df['charges']
```

Train Multiple Linear Regression Model

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = LinearRegression()

# Train model
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Model Evaluation

```
# Predictions
y_pred = model.predict(X_test)

# Evaluation Metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("R2 Score:", r2)
```

```
MAE: 4213.798594527248
MSE: 33981653.95019776
R2 Score: 0.7811147722517886
```

Which Factors Influence Insurance Charges Most?

```
coefficients = pd.DataFrame(model.coef_, X.columns, columns=['Coefficient'])
print(coefficients)
```

```
           Coefficient
age         257.071158
bmi         327.522631
children    427.199971
smoker    23653.211646
```

Lab 14: Multiple Linear Regression Aim

To implement Multiple Linear Regression and evaluate its performance using RMSE and R^2 score. Also compare it with Simple Linear Regression.

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Load Dataset

```
# Load dataset
data = pd.read_csv("/content/Advertising Budget and Sales.csv")

# Display first 5 rows
print(data.head())
```

```
   Unnamed: 0  TV Ad Budget ($)  Radio Ad Budget ($)  Newspaper Ad Budget ($)  \
0           1             230.1                 37.8                     69.2
1           2              44.5                 39.3                     45.1
2           3              17.2                 45.9                     69.3
3           4             151.5                 41.3                     58.5
4           5             180.8                 10.8                     58.4

   Sales ($)
0       22.1
```

```
1       10.4
2        9.3
3       18.5
4       12.9
```

Step 3: Select Features and Target

```
X = data[['TV Ad Budget ($)', 'Radio Ad Budget ($)', 'Newspaper Ad Budget ($)']]
y = data['Sales ($)']
```

Step 4: Split Data (80% Train, 20% Test)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Step 5: Train Multiple Linear Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Step 6: Make Predictions

```
y_pred = model.predict(X_test)
```

Step 7: Evaluate Model Performance

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("Multiple Linear Regression Results:")
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
Multiple Linear Regression Results:
RMSE: 1.78159966153345
R2 Score: 0.899438024100912
```

Step 8: Simple Linear Regression (TV vs Sales)

```
# Use only TV feature
X_simple = data[['TV Ad Budget ($)']]
y_simple = data['Sales ($)']

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_simple, y_simple, test_size=0.2, random_state=42)

simple_model = LinearRegression()
simple_model.fit(X_train_s, y_train_s)

y_pred_s = simple_model.predict(X_test_s)

rmse_s = np.sqrt(mean_squared_error(y_test_s, y_pred_s))
r2_s = r2_score(y_test_s, y_pred_s)

print("\nSimple Linear Regression Results (TV vs Sales):")
print("RMSE:", rmse_s)
print("R2 Score:", r2_s)
```

```

Simple Linear Regression Results (TV vs Sales):
RMSE: 3.194472431998898
R2 Score: 0.6766954295627076
```

Lab 14: Multiple Linear Regression

1. Import Libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

2. Load and Preview Dataset

Download the dataset manually from Kaggle: https://www.kaggle.com/datasets/hellbuoy/car-price-prediction

Then:

```
df = pd.read_csv("/content/CarPrice_Assignment.csv")
df.head()
```

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	enginelocation	wheelbase	...	en
0	1	3	alfa-romero giulia	gas	std	two	convertible	rwd	front	88.6	...	
1	2	3	alfa-romero stelvio	gas	std	two	convertible	rwd	front	88.6	...	
2	3	1	alfa-romero Quadrifoglio	gas	std	two	hatchback	rwd	front	94.5	...	
3	4	2	audi 100 ls	gas	std	four	sedan	fwd	front	99.8	...	
4	5	2	audi 100ls	gas	std	four	sedan	4wd	front	99.4	...	

5 rows × 26 columns

3. Select Numeric Features

```
features = ["enginesize", "horsepower", "citympg"]
X = df[features]
y = df["price"]
```

4. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

5. Train Multiple Linear Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

6. Model Evaluation

```
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("R² Score:", r2)
```

```
Mean Squared Error (MSE): 16539320.75061692
R² Score: 0.790493053605295
```

7. Coefficients & Intercept

```
coeff_df = pd.DataFrame({
    "Feature" : features,
    "Coefficient" : model.coef_
})
```

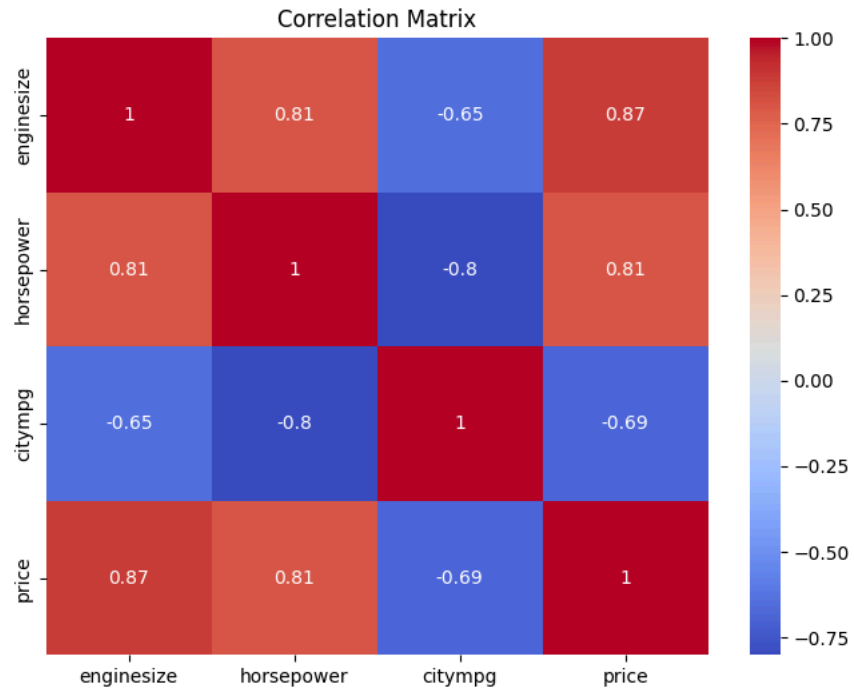
```
print(coeff_df)

print("Intercept:", model.intercept_)
```

```
      Feature  Coefficient
0  enginesize   115.074484
1  horsepower    39.082682
2     citympg  -195.788854
Intercept: -506.64131973569965
```

8. Correlation Matrix (Check Multicollinearity)

```
plt.figure(figsize=(8,6))
corr_matrix = df[features + ["price"]].corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



Lab 14: Multiple Linear Regression Aim

To implement Multiple Linear Regression on a vehicle dataset to predict CO2 Emissions using features like Engine Size, Cylinders, and Fuel Consumption, and evaluate model performance.

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Load Dataset

```
df = pd.read_csv("/content/CO2 Emissions_Canada.csv")
print(df.head())
```

```
    Make       Model Vehicle Class  Engine Size(L)  Cylinders Transmission  \
0  ACURA         ILX       COMPACT             2.0          4          AS5
1  ACURA         ILX       COMPACT             2.4          4           M6
2  ACURA  ILX HYBRID       COMPACT             1.5          4          AV7
3  ACURA     MDX 4WD   SUV - SMALL             3.5          6          AS6
4  ACURA     RDX AWD   SUV - SMALL             3.5          6          AS6

  Fuel Type  Fuel Consumption City (L/100 km)  \
0         Z                               9.9
1         Z                              11.2
2         Z                               6.0
3         Z                              12.7
4         Z                              12.1
```

```
   Fuel Consumption Hwy (L/100 km)  Fuel Consumption Comb (L/100 km)  \
0                              6.7                               8.5
1                              7.7                               9.6
2                              5.8                               5.9
3                              9.1                              11.1
4                              8.7                              10.6

   Fuel Consumption Comb (mpg)  CO2 Emissions(g/km)
0                           33                  196
1                           29                  221
2                           48                  136
3                           25                  255
4                           27                  244
```

Step 3: Select Features and Target

```
X = df[['Engine Size(L)', 'Cylinders', 'Fuel Consumption Comb (L/100 km)']]
y = df['CO2 Emissions(g/km)']
```

Step 4: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 5: Implement Multiple Linear Regression

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Step 7: Evaluate Model RMSE (Root Mean Squared Error)

```
y_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("RMSE:", rmse)
```

```
RMSE: 20.54074808533515
```

```
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

```
R2 Score: 0.8773348735033226
```

Step 8: Interpret Coefficients

```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

print(coefficients)
print("Intercept:", model.intercept_)
```

```
                           Feature  Coefficient
0                    Engine Size(L)     5.594713
1                         Cylinders     6.380798
2  Fuel Consumption Comb (L/100 km)    13.270330
Intercept: 51.52997801495357
```